Contents

1 JReStructor: Java Class Restructuring Tool **3**
- 1.1 Overview . . . 3
- 1.2 Key Advantages . . . 3
- 1.3 Main Flow of the Tool . . . 3

2 Configurator **3**
- 2.1 Purpose . . . 3
- 2.2 Configuration Assembly Flow . . . 3
- 2.3 Key Components (suggested interfaces only) . . . 4
- 2.4 Proposed Core Types . . . 4
  - 2.4.1 1. `OverridingConfiguration` . . . 4
- 2.5 2. `Configuration` . . . 4
- 2.6 Proposed method contracts . . . 5
  - 2.6.1 Resolve Signature . . . 5
  - 2.6.2 Merge Signature . . . 5
- 2.7 Notes . . . 5
- 2.8 Optional Configuration Sources Control . . . 5

3 Java AST Parser **5**
- 3.1 Purpose . . . 5
- 3.2 What is AST (Abstract Syntax Tree)? . . . 6
  - 3.2.1 Example . . . 6
- 3.3 Where It Fits in the Pipeline . . . 6
- 3.4 Candidate Libraries . . . 6
  - 3.4.1 1. JavaParser . . . 6
  - 3.4.2 2. Spoon . . . 6
  - 3.4.3 3. Eclipse JDT AST . . . 7
  - 3.4.4 4. ANTLR with Java grammar . . . 7
- 3.5 Parser Selection Criteria (POC Plan) . . . 7
- 3.6 Additional Notes . . . 7
- 3.7 Summary . . . 7
  - 3.7.1 Primary candidates: . . . 8
  - 3.7.2 Secondary fallback: . . . 8
- 3.8 Related Tools . . . 8

4 Sorter **8**
- 4.1 Purpose . . . 8
- 4.2 What Gets Sorted . . . 8
- 4.3 Input . . . 8
- 4.4 Algorithm Overview . . . 8
- 4.5 Design Considerations . . . 9
- 4.6 Testing Strategy for Fields Resorting . . . 9
- 4.7 Requirements . . . 9

5 Formatter Wrapper **9**
- 5.1 Purpose . . . 9
- 5.2 Role in the Pipeline . . . 10
- 5.3 Formatter Usage . . . 10
- 5.4 Configuration Parameters . . . 10
- 5.5 Summary . . . 10

6 Central Processor **10**
- 6.1 Purpose . . . 10
- 6.2 Dual Flow Overview . . . 11
  - 6.2.1 1. Restructure Flow . . . 11

6.2.2 2. Check Flow . . . 11
6.3 Public API Methods . . . 11
6.3.1 1. Process a single Java source code string . . . 11
6.3.2 2. Process a single Java file . . . 11
6.3.3 3. Process a directory of Java files (recursively, in parallel) . . . 11
6.4 Internal Processing Flow . . . 11
6.5 Extensibility and Flexibility . . . 12
6.6 Check Mode (Validation Without Rewrite) . . . 12
6.6.1 Purpose: . . . 12
6.6.2 Public Methods: . . . 12
6.6.3 Exception Behavior: . . . 12
6.7 Additional Note: Compilation Validation Step . . . 12
6.8 Backup Handling Before File Overwrite . . . 13

7 DiffReporter **13**
7.1 Purpose . . . 13
7.2 Usage . . . 13
7.3 Possible Implementations . . . 13
7.4 Next Steps . . . 13

8 CLI Runner **13**
8.1 Purpose . . . 14
8.2 Responsibilities . . . 14
8.3 Preliminary Argument Set (Subject to Change) . . . 14
8.4 Expected Exit Codes . . . 14
8.5 Implementation Notes . . . 14
8.6 Note . . . 14

9 Maven Plugin: `jrestructor-maven-plugin` **14**
9.1 Purpose . . . 14
9.2 Features . . . 15
9.3 Plugin Goals . . . 15
9.4 Configuration Options . . . 15
9.5 Maven Phase . . . 15
9.6 Sample Usage . . . 15
9.7 Testing Strategy . . . 16
9.8 Notes . . . 16

10 Gradle Plugin (Optional) **16**
10.1 Overview . . . 16
10.2 Purpose . . . 16
10.3 Status . . . 16
10.4 Key Requirements . . . 16
10.5 Contribution Notice . . . 16

11 Testing Strategy **17**
11.1 Overview . . . 17
11.2 Test Coverage Goals . . . 17
11.2.1 Sorter Test Cases . . . 17
11.2.2 Testing Without Fully Working Parser . . . 17
11.3 DiffReporter Testing . . . 17
11.4 Future Considerations . . . 17

12 Documentation **17**
12.1 Repository Structure . . . 18
12.2 Required Markdown Documents . . . 18
12.2.1 `README.md` (root level) . . . 18

12.2.2 /core/README-CORE.md . . . 18
12.2.3 /maven-plugin/README-MAVEN.md . . . 18
12.2.4 /examples/EXAMPLE.md (one per integration) . . . 18
12.3 Jenkins Integration . . . 18
12.3.1 /docs/CONFIG.md . . . 19
12.3.2 /docs/TROUBLESHOOTING.md . . . 19
12.3.3 /docs/CHANGELOG.md . . . 19

1 JReStructor: Java Class Restructuring Tool

1.1 Overview

JReStructor is a modular tool for restructuring Java source code. Its main purpose is to automatically reorder and format members of Java classes (fields, constructors, methods, blocks, etc.) to ensure a consistent and readable structure based on configurable rules.

This tool is especially useful in environments where large teams work on shared codebases and consistency of structure is important for code reviews, quality checks, or compliance with internal style guides.

1.2 Key Advantages

- **Automated Restructuring**: Eliminates the manual effort of sorting class members.
- **Highly Configurable**: Supports a wide range of rules to control the sorting and formatting logic.
- **Check Mode**: Validates whether source files conform to the desired structure without modifying them.
- **AST-based Manipulation**: Uses abstract syntax trees to ensure syntactic correctness.
- **Extensible Architecture**: Designed for integration with Maven, Gradle, and CI/CD pipelines.

1.3 Main Flow of the Tool

The main flow of JReStructor can be summarized as:

1. **Configuration Resolution**: Load user-defined configuration (inline, file, or environment).
2. **Parsing**: Convert Java source into AST using selected parser (e.g., JavaParser, Spoon).
3. **Sorting**: Apply restructuring rules to reorder class members.
4. **Formatting**: Clean up code using an external formatter (e.g., Palantir Java Format).
5. **Diffing (optional)**: In check mode, compare original and modified version for consistency.
6. **Writing (optional)**: In restructure mode, save updated source code to disk or return as string.

Each of these steps is handled by dedicated components, described in detail in the corresponding documentation modules.

2 Configurator

2.1 Purpose

To construct a validated, immutable configuration model (`Configuration`) by:

- loading internal defaults from resources,
- overlaying with external configuration sources (IDE (IDEA, possibly Eclipse), configuration file, passed model from external wrapper),
- resolving priority and applying overrides in sequence.

2.2 Configuration Assembly Flow

1. Load default configuration from embedded resource file
   → `Configuration defaultConfig`

2. Optionally overlay:
    - IDE configuration (optional) - it must be IDEA and possibly Eclipse
    - Custom file configuration (optional)
    - External override (optional)
3. Each overlay produces a merged intermediate:
   `defaultConfig → mergedConfig1 → mergedConfig2 → ... → finalConfig`

2.3 Key Components (suggested interfaces only)

Component	Responsibility
`DefaultConfigLoader`	Loads embedded default configuration from `resources/`.
`IDEConfigParser`	Parses IDE-specific configuration formats (e.g. `.editorconfig`, `.idea/*.xml`).
`ProjectFileConfigParser`	Reads XML configuration from project sources.
`Configurator`	Coordinates the full assembly process; returns validated `Configuration`.

These additional configuration parsers can implement a unified interface `OverridingConfigurationProvider` to be processed as a sequence in a unified stream/cycle algorithm.

2.4 Proposed Core Types

All code snippets given only as illustrations of the idea

2.4.1 1. `OverridingConfiguration`

A nullable, non-validated DTO representing raw configuration input.

Used as intermediary for loading from:

- default resource file
- IDE settings (JetBrains, Eclipse)
- project configuration file (XML/YAML)
- external override models (e.g. plugin input)

```
@Data
public class OverridingConfiguration {
    private List<String> memberOrder;
    private List<String> accessLevelOrder;
    private Integer maxLineLength;
    private Boolean reorderImports;
}
```

2.5 2. `Configuration`

Final, validated, immutable model for internal use.

```
@Value
@Builder
public class Configuration {
    List<String> memberOrder;
    List<String> accessLevelOrder;
    int maxLineLength;
    boolean reorderImports;
}
```

2.6 Proposed method contracts

All code snippets given only as illustrations of the idea

2.6.1 Resolve Signature

```
public interface Configurator {
    Configuration resolve(
        List<OverridingConfiguration> externalConfigs,
        List<Path> configFiles
    );
}
```

2.6.2 Merge Signature

```
public interface Configurator {
   Configuration merge(Configuration baseConfig, OverridingConfiguration overridingConfiguration
}
```

2.7 Notes

- `OverridingConfiguration` can be reused across all stages as the universal nullable intermediate.
- Merging logic should handle null fields safely, always preserving previous values unless explicitly overridden.
- Multiple `merge()` calls are expected in sequence.
- Final `Configuration` must be fully initialized and validated (non-null + correct values).

2.8 Optional Configuration Sources Control

For advanced usage, the configurator supports **optional control over which sources to include** when resolving the final configuration.

By default, all sources are enabled: - IDE configuration (e.g. IntelliJ, Eclipse) - Project-level configuration files (XML/YAML) - External override inputs (e.g. plugin parameters)

However, users can pass boolean flags (e.g. `disableIDEA`, `disableFileConfig`) to suppress specific sources.

Additionally, for full flexibility, it is proposed that the configurator exposes a method accepting a **custom sequence of configuration parsers**, each implementing a common interface (e.g. `RawConfigProvider`).

This allows advanced consumers (e.g. plugins, test suites) to **control exactly which parsers participate** in the configuration resolution process.

```
public interface OverridingConfigurationProvider {
    OverridingConfiguration load();
}
```

Suggested method for customized merging:

```
public Configuration resolve(List<OverridingConfigurationProvider> customProviders);
```

This approach makes it possible to: - Test specific parsers in isolation - Apply only trusted configuration layers (e.g. CI pipelines) - Extend or replace default parsing logic

3 Java AST Parser

3.1 Purpose

In the context of the JReStructor project, we aim to evaluate and select a reliable and flexible Java parser that transforms raw Java source code into an object-oriented model (AST - Abstract Syntax Tree). This is a critical step for performing member reordering and clean reserialization back into Java source. Post-sorting, raw code might

lose its original formatting and readability. Thus, to preserve correctness and maintain format quality, we must rely on a robust parser + formatter pair.

The AST parser must serve as the foundation for: - breaking code into structured elements, - performing sorting based on rules and configurations, - and reserializing it reliably, preserving annotations, comments, formatting consistency, and correct Java syntax.

3.2 What is AST (Abstract Syntax Tree)?

AST (Abstract Syntax Tree) is a tree-like data structure used to represent the **syntactic structure** of source code in a hierarchical way.

Each node in the tree corresponds to a **construct** occurring in the code: - expressions, - variable declarations, - method calls, - class definitions, etc.

3.2.1 Example

For this Java code:

```java
int x = 2 + 3;
```

The corresponding AST structure would look like:

```
Assignment
+- Variable: x
\- Expression: +
    +- Literal: 2
    \- Literal: 3
```

3.3 Where It Fits in the Pipeline

```
Raw Java code
    ↓
AST Parser (e.g., JavaParser or Spoon)
    ↓
Object Model (Class → Fields, Methods, Inner Types, etc.)
    ↓
Sorting and processing
    ↓
Back to source (reserialization)
    ↓
Formatter (Palantir Java Format)
    ↓
Final formatted Java code
```

3.4 Candidate Libraries

3.4.1 1. JavaParser

- Maintained actively, supports Java 1-21
- Simple API for parsing, visiting, and manipulating AST
- Supports pretty-printing with formatting preservation
- Comments and annotations handled explicitly
- Can parse broken/incomplete Java code (with exceptions)

3.4.2 2. Spoon

- Academic-grade deep Java code analysis and transformation framework
- More powerful than JavaParser but more complex

- AST with rich semantic information
- Ideal for deep refactoring, not just basic sorting
- Also supports comments and annotations

3.4.3 3. Eclipse JDT AST

- The AST used by the Eclipse IDE
- Heavy, low-level API
- Requires Eclipse infrastructure
- Not ideal for lightweight standalone tools

3.4.4 4. ANTLR with Java grammar

- Manual integration with grammar definitions
- Error-prone and requires high effort
- Not considered suitable for our primary flow

3.5 Parser Selection Criteria (POC Plan)

We must test: 1. **Valid Java Source** - Parse a valid `.java` file - Check resulting AST structure - Evaluate reserialization output

2. **Broken Java Source**
   - Test malformed files
   - Determine how gracefully parser fails
   - Expected: structured error or partial AST
3. **Comments Preservation**
   - Block, inline, and Javadoc comments
   - Must survive roundtrip (parse + output)
4. **Annotations**
   - On fields, methods, classes
   - Preservation + *ability to sort annotations*
5. **Inner/Nested Classes**
   - Must retain proper nesting in AST and output
   - Especially test deep nesting
6. **Field Dependency Sorting**
   - Determine order based on other field usage
   - Does AST contain information about field dependencies?
7. **Reserialization Quality**
   - Check fidelity of output
   - Formatting errors or losses?
8. **Java Version Compatibility**
   - Ensure Java 21 features are parsed correctly

3.6 Additional Notes

- We may wrap the parser of choice in a lightweight adapter component to abstract its API and expose a simplified JReStructor-specific interface.
- This component will serve as the core input/output layer for sorting and transformation logic.

3.7 Summary

The selected parser must: - Be easy to use and integrate - Provide good support for annotations and comments - Work with Java 21 syntax - Tolerate broken code inputs gracefully - Output clean, faithful source via serialization

3.7.1 Primary candidates:

- JavaParser (Preferred starting point)
- Spoon (Alternative with deeper insight)

3.7.2 Secondary fallback:

- Eclipse JDT AST (fallback)
- ANTLR (fallback)

If needed, fallback parser integration may be considered for future iterations depending on POC results and resource availability.

3.8 Related Tools

- JavaParser
- Eclipse JDT AST
- ANTLR for Java

4 Sorter

4.1 Purpose

This document describes the design and responsibilities of the member sorting component in the JReStructor tool. The purpose of the sorter is to provide consistent, configurable ordering of all class members in a Java source file, to ensure readability, maintainability, and stability of diffs.

4.2 What Gets Sorted

The sorter is responsible for ordering the following elements within a Java class:

- Fields
- Initializer blocks (static and instance)
- Constructors
- Methods (static and instance)
- Inner classes and interfaces

4.3 Input

The sorter operates on:

- An **AST model** of a parsed Java class (produced by a separate parser step).
- A **Configuration** object containing:
  - Expected member ordering (e.g. fields → constructors → methods)
  - Visibility ordering (e.g. public → protected → package-private → private)
  - Sorting rules within categories (e.g. alphabetical)
  - Options for nested class processing
  - Special cases: getter/setter pairs, constructors, initialization blocks

4.4 Algorithm Overview

1. **Group Members by Type**: All members are classified (fields, methods, etc.) and grouped accordingly.
2. **Recursive Processing of Inner Classes**: The sorter recursively applies itself to inner classes using the same configuration.
3. **Sort All Categories Except Fields**: Standard sorting is applied:

- First by visibility
- Then by alphabetic name (if configured)

4. **Special Handling for Fields**: Fields may depend on each other:

   ```
   int b = 42;
   int a = b + 1;
   ```

   In this case, b must appear **before** a, regardless of alphabetical or visibility preferences.

   The algorithm must:

   - Build a **dependency graph** of fields
   - Attempt to apply desired order **without violating dependency constraints**
   - If conflicts arise, prioritize correctness and preserve original order as fallback

5. **Output**:

   - An updated AST with members sorted according to the resolved order.

4.5 Design Considerations

- **Field Dependencies** are the most complex challenge.
- The algorithm must be **deterministic** and repeatable.

4.6 Testing Strategy for Fields Resorting

Even before final parser selection, the sorting logic can be prototyped using in-memory mock models that simulate class members and their relationships. This allows testing of:

- Sorting correctness
- Dependency resolution
- Corner cases like:
  - Circular dependencies
  - Annotated fields/methods
  - Static and non-static member interleaving

4.7 Requirements

- Recursively support nested classes
- Fully configurable via `Configuration`
- Operate on an AST model (from JavaParser or Spoon, etc.)
- Compatible with Java 21 constructs

5 Formatter Wrapper

5.1 Purpose

The formatter wrapper integrates the Palantir Java Formatter into the `JReStructor` toolchain to ensure clean and consistent Java code output **after deserialization** of an AST structure.

This step is **critical** because deserialized Java code may lose indentation, import order, and other formatting features. Formatter ensures:

- Correct indentation
- Removal and ordering of imports
- Application of consistent code style (Palantir, Google, AOSP)

5.2 Role in the Pipeline

```
Java source (.java)
    ↓
Parse to AST model
    ↓
Sort and transform AST
    ↓
Serialize AST back to Java code
    ↓
Format output using Palantir formatter
```

5.3 Formatter Usage

The wrapper internally delegates to this verified method from `com.palantir.javaformat.java.Formatter`:

```java
/**
 * Formats an input string (a Java compilation unit) and fixes imports.
 *
 * Fixing imports includes ordering, spacing, and removal of unused import statements.
 *
 * @param input the input string
 * @return the output string
 * @throws FormatterException if the input string cannot be parsed
 */
public String formatSourceAndFixImports(String input) throws FormatterException
```

5.4 Configuration Parameters

Formatter style is configured via `JavaFormatterOptions.Style`, with three supported styles:

```java
public enum Style {
    /** The default Palantir Java Style configuration. */
    PALANTIR(2, 120),

    /** The default Google Java Style configuration. */
    GOOGLE(1, 100),

    /** The AOSP-compliant configuration. */
    AOSP(2, 100);
}
```

These parameters must be passed through the `Configuration` model and injected into the formatter via our `FormatterWrapper`.

5.5 Summary

This wrapper is essential to finalize formatted, readable Java source code that adheres to a unified style. It should be executed only **after** AST transformation and serialization are complete.

6 Central Processor

6.1 Purpose

`Processor` is the main orchestrator of the system. It performs configuration aggregation first, then manages the full transformation pipeline: - Parsing Java source code into an abstract syntax tree (AST) - Sorting class members

according to the specified configuration - Serializing the sorted AST back to source code - Formatting the final source code using `PalantirJavaFormat`.

6.2 Dual Flow Overview

The Processor supports two distinct execution flows:

6.2.1 1. Restructure Flow

- Full processing pipeline: config → parse → sort → serialize → format
- Produces a new source code
- Optionally writes to file(s)
- Can return stats (count, size, duration)

6.2.2 2. Check Flow

- Same pipeline as above, but only in memory
- Compares input and output
- Throws if restructuring would alter content
- Used for CI/linting

6.3 Public API Methods

6.3.1 1. Process a single Java source code string

```
String restructure(String inputJavaCode, OverridingConfiguration configuration, Path configFile,
```

- **Input**:
  - Java source code string
  - Configuration input (`OverridingConfiguration`)
  - Flags indicating which config parsers are enabled/disabled
- **Output**: Transformed and formatted Java code string

6.3.2 2. Process a single Java file

```
FileProcessingStatisitic restructure(Path inputFilePath, OverridingConfiguration configuration,
```

- Reads file content, processes it, and overwrites or replaces the original file
- **Output**: Processing statistic: processing time, size before and after transformation, error count, etc.

6.3.3 3. Process a directory of Java files (recursively, in parallel)

```
ProcessingReport restructure(Path inputDirectory, OverridingConfiguration configuration, Path co
```

- Recursively processes `.java` files in the directory
- Executes processing in parallel threads
- **Output**: `ProcessingReport` containing:
  - Number of processed files
  - Total and average processing time
  - Min/max file size
  - Error count
  - etc.

6.4 Internal Processing Flow

1. **Configure**: All entry points first invoke `Configurator` to collect and merge all configuration sources into a final `Configuration`.
2. **Parse**: Java source is parsed into an AST via a chosen parser (e.g., `JavaParser`, Spoon).

3. **Sort**: Members of the AST are reordered using `ASTSorter` based on configuration.
4. **Serialize**: AST is converted back to Java source code.
5. **Format**: Final code is formatted using `formatSourceAndFixImports()` with the appropriate style (Palantir/Google/AOSP).

6.5 Extensibility and Flexibility

- **Multi-source configuration support**: YAML, IDE settings, inline configs.
- **Parser control flags**: selectively enable/disable configuration sources.
- **Contextual outputs**: method output varies depending on target (string, file, directory).
- **Metrics & logging hooks**: optionally integrated for performance tracking and diagnostics.

6.6 Check Mode (Validation Without Rewrite)

In addition to full restructuring, the Processor also provides a **`check` mode**.
This flow performs the same pipeline (configuration → parsing → sorting → serialization → formatting) but **only in memory** and does **not write any changes**.
It compares the original input with the result and throws an exception if restructuring would make changes.

6.6.1 Purpose:

Used in CI or quality gates to validate whether code is already correctly structured.

6.6.2 Public Methods:

All configuration parameters were omitted for simplicity.

- `void check(Path directory)`
  - Recursively checks all `.java` files in a directory.
  - If any file differs from its restructured version, throws `CodeNotRestructuredException`.
- `void check(Path file)`
  - Checks a single file.
- `void check(String javaSource)`
  - Checks a raw Java source string.

6.6.3 Exception Behavior:

If restructuring changes the code, a `CodeNotRestructuredException` is thrown.
The exception should contain: - Path or origin description - Unified diff for diagnostics

6.7 Additional Note: Compilation Validation Step

In certain cases, if the selected AST parser fails to clearly identify invalid Java source files (e.g., files with syntactical or structural corruption), it may be necessary to integrate a lightweight and embeddable Java compiler into the processing pipeline.

The idea is to pre-validate each file by attempting to compile it before parsing and restructuring. This compilation phase can serve as a safeguard to ensure that the file is valid Java code. If the compiler fails with a meaningful error message, it can prevent the restructuring pipeline from executing on a broken file and help log or report the cause of failure.

This step may be performed in parallel or as a pre-processing phase before invoking the AST parser. The need for this step will depend on the behavior of the selected parser and should be evaluated after initial POC validation.

6.8 Backup Handling Before File Overwrite

To prevent accidental data loss when overwriting files during restructuring, especially in environments where no version control is used, implement an optional backup mechanism.

- **Condition**: A backup file should be created **only if** the output differs from the original input (i.e. the file is modified).
- **Backup naming convention**: Append a `.bak` or `.backup` extension to the original filename (e.g. `MyClass.java.bak`).
- **Activation**: This feature should be configurable via `Configuration`, e.g. a flag like `createFileBackup = true`.
- **Rationale**: Enables safe recovery if something goes wrong during restructuring, or if a user prefers to manually inspect changes.

7 DiffReporter

7.1 Purpose

In **check mode**, a dedicated component — `DiffReporter` — is required to compare the **original Java source code with the restructured version**. It serves two key purposes:

1. **Comparison**: Determines whether the original and transformed code are identical.
2. **Diagnostics**: If differences exist, generates a readable **diff output** suitable for terminal or log output, highlighting the changes clearly.

7.2 Usage

`DiffReporter` is invoked from within the `check(...)` method to: - Validate whether restructuring is necessary. - Raise an exception when mismatches are found, including a detailed diff report.

7.3 Possible Implementations

1. **Existing Java libraries**:
   - `google-diff-match-patch`
   - `java-diff-utils`
2. **Reviewing Palantir Java Formatter**:
   - Although `palantir-java-format` does not expose a standalone diff component, its internal logic in **`check` mode** can serve as a valuable **source of inspiration**. Investigating how it compares the formatted output with the original may offer reusable strategies.
3. **Custom implementation**:
   - Line-by-line comparison with highlighted differences;
   - Optional highlighting at the character level;
   - Configurable verbosity (e.g., control display of whitespace-only differences or empty lines).

7.4 Next Steps

- Conduct a technical research sweep:
  - Benchmark available libraries in terms of performance, Unicode support, and diff quality.
  - Explore the comparison logic used in `palantir-java-format`.
  - If necessary, implement a minimal internal `DiffReporter` prioritizing readability and extensibility.

8 CLI Runner

Draft Proposal
This document outlines a preliminary sketch for adding command-line capabilities to the restructuring

tool.
It is not a finalized design and should be refined during implementation.

8.1 Purpose

Introduce a lightweight, console-invocable entry point that allows triggering the entire restructuring pipeline via standard terminal commands.
This CLI component is aimed at: - Supporting quick local testing and debugging - Allowing integration into CI/CD and automation workflows - Providing a developer-friendly way to validate or restructure files without needing to embed the tool into another Java application

8.2 Responsibilities

- Parse CLI arguments
- Prepare and trigger configuration resolution
- Instantiate the main Processor and call either `restructure` or `check` flow
- Report outcome via terminal messages and exit codes

8.3 Preliminary Argument Set (Subject to Change)

Argument	Description
`--mode=`	Operation mode: `restructure` or `check`
`--input=`	File or directory path to be processed
`--config=`	Path to configuration file(s)
`--flags=`	Optional override flags (e.g. `parser1:on,parser2:off`)

8.4 Expected Exit Codes

Code	Meaning
0	Success (no changes needed or restructure done)
1	Failure (e.g. check mode failed due to mismatch)
2+	Errors: invalid args, IO issues, internal crash

8.5 Implementation Notes

- Prefer lightweight CLI parsers (e.g. `picocli`, `JCommander`) for ease of maintenance.
- It should stay thin: focus only on delegation, logging, and error handling.

8.6 Note

Further design will evolve once configuration bootstrapping and Processor contracts are finalized.

9 Maven Plugin: `jrestructor-maven-plugin`

9.1 Purpose

This Maven plugin is designed to integrate the **JReStructor** utility into Java projects as a build phase tool. It enables automated restructuring or structure validation of Java source files directly from Maven, with flexible configuration and execution modes.

9.2 Features

- Run structure **reformatting** or **check-only validation** as part of the Maven lifecycle.
- Configurable to operate on `src/main/java` by default, with an option to include `src/test/java`.
- Three validation severity levels for `check` mode.
- Executes before source code generation phase to prevent working on already generated code.
- Accepts inline configuration via `<configuration>` section in `pom.xml`.
- Supports backup of modified files if desired.

9.3 Plugin Goals

- `jrestructor:check` — validates whether files are already properly structured.
- `jrestructor:restructure` — restructures Java source files according to the defined sorting and formatting logic.

9.4 Configuration Options

Parameter	Type	Description
`mode`	`String`	Either `check` or `restructure`.
`includeTestSources`	`boolean`	Whether to include `src/test/java` in addition to `src/main/java`.
`severityLevel`	`String`	`fail-fast`, `collect-and-fail`, or `warn-only` for check mode.
`configFiles`	`List`	Optional paths to config files to override defaults.
`overrideConfigs`	`List`	Optional list of inline configuration overrides.
`parserFlags`	`List`	Optional parser customization flags.
`enableBackup`	`boolean`	Whether to create backups before overwriting any files.

9.5 Maven Phase

By default, the plugin is configured to execute before `generate-sources`, ensuring that only manually written Java files are processed.

9.6 Sample Usage

```
<plugin>
  <groupId>com.example</groupId>
  <artifactId>jrestructor-maven-plugin</artifactId>
  <version>0.1.0</version>
  <executions>
    <execution>
      <goals>
        <goal>check</goal>
      </goals>
      <phase>generate-sources</phase>
    </execution>
  </executions>
  <configuration>
    <mode>check</mode>
    <includeTestSources>true</includeTestSources>
    <severityLevel>collect-and-fail</severityLevel>
    <enableBackup>true</enableBackup>
  </configuration>
</plugin>
```

9.7 Testing Strategy

The plugin will use the **Maven Plugin Testing Framework** (e.g. `org.apache.maven.plugin.testing`) or alternatives such as **Invoker Plugin** for full integration testing.

Tests will:

- Ensure valid detection and behavior under all severity levels.
- Verify file changes and backup creation.
- Confirm plugin integration works across multi-module projects and test configurations.

9.8 Notes

This document is a **draft specification**. Some configuration keys and plugin behaviors may be refined during implementation.

10 Gradle Plugin (Optional)

10.1 Overview

While the core focus of this project is on delivering a reliable and tested Maven plugin for integration with Java projects, we acknowledge the widespread use of Gradle in modern Java ecosystems. Therefore, we define the development of a **Gradle plugin** as an **optional contribution**, outside the primary scope of the hackathon deliverables.

10.2 Purpose

The Gradle plugin would mirror the functionality provided by the Maven plugin. It would allow Java projects using Gradle to integrate the JReStructor tool directly into their build pipeline, enabling:

- Automatic restructuring (`restructure`) of Java source code during the build process.
- Validation (`check`) of formatting consistency to prevent unformatted code from passing through CI/CD pipelines.
- Custom configuration passed through Gradle's extension and plugin DSL.

10.3 Status

- **Optional**: The Gradle plugin is **not required** for the hackathon MVP.
- **Open for Contribution**: If team members experienced in Gradle development express interest, they are welcome to take ownership of this component.
- **Testing Strategy**: Similar to Maven, the plugin should be validated through representative Gradle test projects and integration testing.

10.4 Key Requirements

- Provide Gradle DSL extension for configuration options (e.g. included paths, check/restructure mode, severity settings).
- Hook into appropriate Gradle build lifecycle phases (e.g. `compileJava` or earlier).
- Output diagnostics and reformatted code, or fail builds based on configurable thresholds.

10.5 Contribution Notice

If a contributor or a team member is passionate about Gradle and wishes to implement this plugin, we will fully support the effort. Otherwise, the plugin remains out of scope for initial development.

11 Testing Strategy

11.1 Overview

Testing the core logic of JReStructor can be done in parallel with the development of other components. This includes both the sorter (restructuring logic) and the diff-reporter component.

11.2 Test Coverage Goals

11.2.1 Sorter Test Cases

- **Valid Java classes**:
  - Standard ordering with fields, methods, constructors, static blocks.
- **Corner Cases**:
  - Inner/nested classes.
  - Static initializers before or after fields.
  - Complex field interdependencies (e.g., constant expressions).
  - Interfaces with default methods.
- **Invalid Inputs**:
  - Corrupted or unparseable Java files.
  - Structural syntax issues.

Tests should validate: - Successful restructuring for valid cases. - Proper exception handling for invalid files. - Consistent output according to defined sorting rules.

11.2.2 Testing Without Fully Working Parser

Although the actual restructuring depends on a working parser, the following can be developed **in parallel**: - Construction of test cases. - A test runner to process test files and compare results. - Expected results prepared as `.expected.java` files for comparison.

Stub/mock implementations of the parser can be used to simulate flow during early development.

11.3 DiffReporter Testing

The `DiffReporter` can and **should** be tested independently: - Input: `originalCode.java` and `restructuredCode.java` - Output: Boolean match and human-readable diff.

Unit tests should cover: - Minor structural differences → accurate diff - Major differences → clear summary and line-by-line changes

This ensures robustness of the `check()` mode used in CI pipelines.

11.4 Future Considerations

- Runtime performance benchmarks on large codebases.
- Compatibility testing across Java versions.

12 Documentation

This section defines the expected structure of the documentation and repository layout for the project. It is a non-functional but **mandatory** requirement aimed at ensuring clarity, consistency, and ease of collaboration within the team.

12.1 Repository Structure

```
my-project/
+-- core/                   # Core utility
|   \-- README-CORE.md      # Core documentation
+-- maven-plugin/           # Maven plugin
|   \-- README-MAVEN.md     # Plugin documentation
+-- gradle-plugin/          # Gradle plugin (optional)
|   \-- README-GRADLE.md
+-- examples/               # Example setups
|   +-- jenkins/            # Jenkins example
|   |   \-- EXAMPLE.md      # Description and usage
|   \-- gitlab-ci.yml       # GitLab CI config
+-- docs/                   # Global documentation
|   +-- CONFIG.md           # Configuration
|   +-- TROUBLESHOOTING.md
|   \-- CHANGELOG.md
\-- README.md               # Main entry point
```

12.2 Required Markdown Documents

12.2.1 `README.md` (root level)

Contains: - Project description: *"Code formatting tool + plugins"* - Module links: `markdown - [Core Utility](/core/README-CORE.md) - [Maven Plugin](/maven-plugin/README-MAVEN.md) - [Examples](/examples/)` - Quick start: `bash git clone https://github.com/your/project.git cd project mvn install`

12.2.2 `/core/README-CORE.md`

Contains: - CLI usage: `your-util format --target=src/` - Configuration: `~/.your-util/config.yaml` - Example pipeline: `yaml steps: - name: Format code run: your-util format --check`

12.2.3 `/maven-plugin/README-MAVEN.md`

Contains: - Usage in pom.xml: `xml <plugin> <groupId>com.your</groupId> <artifactId>formatter-maven-plugin</artifactId> <version>1.0</version> </plugin>` - Execution: `mvn formatter:format` - Parameters: `<skip>false</skip>`

12.2.4 `/examples/EXAMPLE.md` (one per integration)

Example: Jenkins

12.3 Jenkins Integration

```groovy
pipeline {
  stages {
    stage('Format') {
      steps {
        withMaven(maven: 'maven-3.8') {
          sh 'mvn formatter:format'
        }
```

```
        }
    }
  }
}
```

12.3.1 /docs/CONFIG.md

Contains: - Shared configuration principles - Environment variables: `bash export FORMATTER_DEBUG=true` - Config formats: YAML / JSON

12.3.2 /docs/TROUBLESHOOTING.md

Error	Solution
Unsupported Java version	Upgrade JDK to 17+
Config not found	Run `your-util init` to create default config

12.3.3 /docs/CHANGELOG.md

Example entry:

```
## [1.1.0] - 2025-06-20
### Added
- Java 21 support
```